TECHNICAL DESIGN DOCUMENT

Anansi

AI-Powered Multimodal Education for African Classrooms

Field	Value
Project name	Anansi
Team	SQUAD3 Wakanda
Version	1.0 — Design Phase
Document type	Technical Design Document
Status	In Design
Date	2026

Table of Contents

1. Project Overview

Anansi is an AI-powered multimodal educational agent designed specifically for African primary school classrooms. Given a topic, a country, a grade level, and a target language, Anansi generates a complete teaching package: a multi-panel culturally accurate cartoon, panel-by-panel audio narration in the local language, a printable PDF, discussion cards, and a teacher guide.

The name Anansi comes from Akan and West African folklore — the spider deity who is the keeper of all stories and knowledge. A tool that weaves concepts into visual narratives for African children could have no more fitting a name.

1.1 Mission

To make every AI-generated educational cartoon feel like it was made for that specific child's classroom — with names, places, food, housing, and landscapes drawn from their country and culture, not from a generic Western template.

1.2 Target Users

- Primary school teachers across Sub-Saharan Africa and the Maghreb
- Schools with limited access to professionally designed visual teaching materials
- Educators working in multilingual contexts where English or French is not the primary language of instruction

1.3 Key Design Principles

1. Cultural authenticity — every visual and textual detail is drawn from a real, country-specific cultural context pack
2. Teacher-first — three simple inputs, one complete teaching package
3. Offline-ready — architecture designed so cloud APIs can be swapped to local models with a single config flag
4. Modular and observable — each agent node is independently testable and fully traced via Langfuse
5. Cost-conscious — estimated $0.24 per complete cartoon at Phase 1 scale

2. System Architecture

Anansi is built as a LangGraph stateful agent graph with six nodes, a Streamlit teacher interface, and a FastMCP cultural context server. All inter-node communication happens through a shared TypedDict graph state object.

2.1 High-Level Architecture

Layer	Technology	Responsibility
UI	Streamlit	Teacher input form, live progress streaming, panel viewer, audio playback, download
Agent core	LangGraph	6-node stateful graph, parallel execution, shared state management
Context server	FastMCP (Python)	Cultural context API — 5 tools returning per-country context packs
Image generation	FLUX.1 Kontext Pro	Multi-panel cartoon generation with character consistency
Audio narration	Google Cloud TTS	Panel-by-panel narration in local African languages
Observability	Langfuse	Full trace, prompt analytics, cost and latency per node
Data validation	Pydantic	Structured outputs at every node boundary
Content safety	LLM-as-judge (Haiku)	Age and cultural appropriateness validation

2.2 Graph State

All nodes read from and write to a shared TypedDict state. The state is the single source of truth throughout the pipeline.

```
class AnansiState(TypedDict):
    # Inputs
    topic: str
    country: str
    grade: str
    language: str
    extra_context: str

    # Node 2 output
    context_pack: ContextPack

    # Node 3 output
    scenes: List[Scene]
    panel_scripts: List[PanelScript]

    # Node 4 output (parallel)
    image_urls: List[str]

    # Node 6 output (parallel)
    audio_urls: List[str]
    narrator_script: str

    # Node 5 output
    panels: List[Panel]
    teacher_guide: TeacherGuide
```

2.3 Node Execution Flow

Nodes 1, 2, and 3 execute sequentially. After Node 3 completes, Nodes 4 and 6 execute in parallel (LangGraph parallel branches). Node 5 waits for both to complete before final assembly.

Node	Name	Type	Depends on	Produces
N1	Concept Analyzer	LLM (Sonnet)	Inputs	scenes: List[Scene]
N2	Localizer	LLM + MCP tools	N1	context_pack: ContextPack
N3	Scriptor	LLM (Haiku)	N1, N2	panel_scripts: List[PanelScript]
N4	Cartoon Generator	FLUX Kontext API	N3 (parallel)	image_urls: List[str]
N6	Narrator	Google TTS API	N3 (parallel)	audio_urls: List[str]
N5	Synthesizer	LLM (Haiku)	N4, N6	panels, teacher_guide

3. Agent Node Specifications

3.1 Node 1 — Concept Analyzer

MODEL

Claude Sonnet — used here because concept decomposition benefits from stronger reasoning quality.

INPUT

- topic (str): The educational topic
- grade (str): Target grade or age range
- extra_context (str): Optional teacher-provided context

OUTPUT SCHEMA

```
class Scene(BaseModel):
    panel_number: int
    description: str      # What happens in this panel
    key_concept: str      # The learning point
    characters: List[str] # Characters present
    setting: str          # Where the scene takes place

class StoryboardOutput(BaseModel):
    title: str
    scenes: List[Scene]   # Typically 4-6 panels
    total_panels: int
```

PROMPT STRATEGY

- Deconstruct topic into sequential scenes that build understanding progressively
- Keep scenes visual-first — each scene should be describable as a single image
- Adapt vocabulary and conceptual depth to the grade level
- Leave character names and place names as placeholders (e.g. [CHILD_NAME], [LOCAL_RIVER])

3.2 Node 2 — Localizer

MODEL

Claude Haiku + 5 FastMCP tool calls. The LLM orchestrates the tool calls and assembles the context pack.

MCP TOOL CALLS

Tool	Returns	Data source
get_names(country)	List[str] male, List[str] female	Curated JSON per country
get_places(country)	cities, rivers, landmarks, regions	GeoNames + curated JSON
get_culture(country)	food, clothing, housing, transport, animals	Curated JSON + Wikipedia
get_language(country)	official language, local languages, script	Static language map
get_avoids(country)	List[str] — foreign refs to exclude	Manually curated per country

OUTPUT SCHEMA

```
class ContextPack(BaseModel):
    language: str
    character_names: List[str]
    place_names: List[str]
    cultural_elements: CultureElements
    avoids: List[str]
    art_style_cues: str
```

3.3 Node 3 — Scriptor

MODEL

Claude Haiku — structured output generation is well within Haiku's capability.

OUTPUT SCHEMA

```
class PanelScript(BaseModel):
    panel_number: int
    caption: str          # Max 20 words, age-appropriate
    dialogue: List[dict]  # [{character: str, line: str}]
    image_prompt: str     # Full FLUX prompt, uses context pack
    narration_text: str   # Text for TTS narration

class ScriptOutput(BaseModel):
    panels: List[PanelScript]
    full_narration: str   # Complete narration script
```

IMAGE PROMPT CONSTRUCTION

- Image prompts explicitly include cultural cues from the context pack
- The avoids list is appended as negative guidance: 'Do not show: [avoids list]'
- First panel generates a reference character description reused for consistency

3.4 Node 4 — Cartoon Generator (parallel)

MODEL

FLUX.1 Kontext Pro via SiliconFlow or Replicate API. Cost: approximately $0.04 per image.

CHARACTER CONSISTENCY STRATEGY

- Panel 1 generates freely and saves the result as a reference image
- Panels 2+ include the Panel 1 image as a reference via FLUX Kontext's reference image parameter
- This maintains consistent character appearance across all panels

RETRY LOGIC

- If an image generation fails, retry once with a simplified prompt
- If retry fails, Node 5 uses a placeholder and logs the failure to Langfuse

3.5 Node 6 — Narrator (parallel)

MODEL

Google Cloud Text-to-Speech API. Chosen for its superior coverage of African languages including Swahili, Yoruba, Hausa, Amharic, Zulu, and French dialects spoken in Africa.

OUTPUT MODES

- Panel-by-panel: one audio file per panel, used for synchronized classroom playback
- Full narration: single MP3 file for the complete cartoon, for download

LANGUAGE SELECTION

The language is set from the teacher input and validated against the country context pack (get_language). If the requested language is not available in Google TTS for the selected country, it falls back to the country's official language.

3.6 Node 5 — Synthesizer

MODEL

Claude Haiku — assembles all outputs, performs final review, generates teacher guide.

RESPONSIBILITIES

- Combine panel images, captions, dialogue, and audio URLs into Panel objects
- Generate teacher guide: vocabulary list, comprehension questions, suggested lesson plan
- Perform final LLM-as-judge check on assembled content
- Prepare download package metadata

4. FastMCP Cultural Context Server

4.1 Design

The FastMCP server is a standalone Python service that exposes cultural context data as MCP tools. It is called exclusively by Node 2 (Localizer) at runtime. The server is built with FastMCP and uses simple decorated Python functions returning Pydantic models.

> *The server is intentionally decoupled from the main LangGraph application. It can be updated, extended, or replaced independently without touching the agent graph.*

4.2 Data Architecture

Phase 1 uses static JSON files per country stored in a data/ directory. Each file follows a consistent schema. This approach is simple, version-controllable, and editable by non-developers.

```
data/
  kenya.json
  nigeria.json
  senegal.json
  ghana.json
  cameroon.json
```

Each country JSON file follows this schema:

```
{
  "country": "Kenya",
  "languages": { "official": "English", "local": ["Swahili"], "tts_code":
"sw-KE" },
  "names": { "male": ["Kamau", "Otieno", ...], "female": ["Achieng",
"Wanjiru", ...] },
  "places": { "cities": [...], "rivers": [...], "landmarks": [...] },
  "culture": { "food": [...], "clothing": [...], "housing": [...],
"transport": [...], "animals": [...] },
  "art_style_cues": "East African setting, red soil, acacia trees, ...",
  "avoids": ["snow", "oak trees", "dollars", "subway", ...]
}
```

4.3 Tool Definitions

```
from fastmcp import FastMCP
from pydantic import BaseModel

mcp = FastMCP('anansi-context')

@mcp.tool()
def get_names(country: str) -> NamesResult:
    """Return male and female names for a given African country."""
    ...

@mcp.tool()
def get_places(country: str) -> PlacesResult:
    """Return cities, rivers, and landmarks for a given country."""
    ...
```

4.4 Data Sources

Data type	Source	Update frequency
Place names (cities, rivers, landmarks)	GeoNames API (one-time extraction)	Quarterly

Data type	Source	Update frequency
Cultural context (food, clothing, etc.)	Curated JSON + Wikipedia scrape	On correction submission
Names by country / ethnicity	Curated JSON + community review	On correction submission
Language codes and TTS availability	Google TTS documentation	On new language support
Avoids list	Manually curated per country	On teacher feedback

4.5 Upgrade Path

- Dependencies managed with uv — pyproject.toml and uv.lock define the full environment
- Phase 2: SQLite database, same tool interface — no LangGraph changes required
- Phase 3: PostgreSQL with teacher correction submissions — same tool interface

> *Because the FastMCP tool interface is stable, the data layer can be upgraded without any changes to the LangGraph graph or the Streamlit UI.*

5. Structured Outputs and Guardrails

5.1 Structured Outputs

Every LLM node uses LangChain's .with_structured_output() with a Pydantic model. This ensures clean, validated data flows between nodes rather than unstructured text that requires parsing.

Node	Output model	Key fields
N1 Concept Analyzer	StoryboardOutput	scenes: List[Scene], total_panels: int
N2 Localizer	ContextPack	names, places, cultural_elements, avoids
N3 Scriptor	ScriptOutput	panels: List[PanelScript], full_narration: str
N5 Synthesizer	FinalOutput	panels: List[Panel], teacher_guide: TeacherGuide

5.2 Input Validation

Teacher inputs are validated with Pydantic before the graph starts. Invalid inputs return an error to the Streamlit UI immediately, without consuming any LLM tokens.

```
class TeacherInput(BaseModel):
    topic: str = Field(..., min_length=3, max_length=200)
    country: str = Field(..., description='Must be a supported African
country')
    grade: str
    language: str
    extra_context: str = Field('', max_length=500)

    @validator('country')
    def must_be_supported(cls, v):
        if v not in SUPPORTED_COUNTRIES:
            raise ValueError(f'{v} is not yet supported')
        return v
```

5.3 Content Safety — LLM-as-Judge

After Node 3 generates panel scripts, a dedicated safety check runs using Claude Haiku before image generation begins. This is intentionally an African-context-aware prompt, not a generic Western content filter.

```
SAFETY_PROMPT = '''
You are a content reviewer for African primary school educational materials.
Review this panel script for a Grade {grade} student in {country}.

A description of traditional farming, local food, cultural dress,
or regional customs is ALWAYS appropriate.

Check for:
- Age-appropriate language and scene content
- Cultural respect and accuracy
- Educational accuracy for the topic

Return: {appropriate: bool, issues: List[str], suggestions: List[str]}
'''
```

5.4 Image Prompt Validation

Before each FLUX API call, the generated image prompt is validated to ensure it does not contain items from the country's avoids list and includes at least one locally specific element.

```
class ImagePromptGuard(BaseModel):
    prompt: str
```

```
    contains_avoided_terms: bool   # Must be False
    contains_local_elements: bool  # Must be True
    is_age_appropriate: bool       # Must be True
```

6. Observability — Langfuse

6.1 What is Traced

Trace element	What it captures	Primary use
Request trace	Full execution from input to output	Debugging, latency analysis
Node spans	Per-node timing, input, output, tokens	Cost breakdown, bottleneck identification
LLM calls	Prompt version, model, token count, cost	Prompt optimization
MCP tool calls	Tool name, country input, response	Localizer quality monitoring
Image generation	Prompt sent, model, latency, cost	Image quality tracking
Audio generation	Text sent, language, latency, cost	TTS coverage tracking
Safety check results	Score, issues flagged, action taken	Content quality monitoring

6.2 Key Metrics to Monitor

- Total latency per request (target: under 45 seconds for 5 panels)
- Cost per request by country and grade (identify expensive combinations)
- Image generation failure rate by country (prompt quality indicator)
- MCP tool call success rate (data coverage indicator)
- Safety check flag rate (content quality indicator)
- Teacher feedback rate by country (cultural accuracy indicator)

6.3 Integration

Langfuse integrates via the LangChain callback system — a single CallbackHandler is passed to the graph at invocation time. No instrumentation code is needed inside individual nodes.

```
from langfuse.callback import CallbackHandler

langfuse_handler = CallbackHandler(
    public_key=os.getenv('LANGFUSE_PUBLIC_KEY'),
    secret_key=os.getenv('LANGFUSE_SECRET_KEY'),
)

result = graph.invoke(
    state,
    config={'callbacks': [langfuse_handler]}
)
```

7. Offline Migration Path

7.1 Design Intent

The architecture is explicitly designed so that the transition from cloud APIs to locally hosted models requires a single environment variable change. No graph nodes, no MCP tools, and no Streamlit UI code needs to change.

7.2 Component Swap Table

Component	Phase 1 (Cloud)	Phase 3 (Offline)	Config flag
Text LLM (N1,N3,N5)	Claude Haiku / Sonnet	Ollama (Llama 3.2 / Mistral)	USE_LOCAL=true
Text LLM (N2 Localizer)	Claude Haiku	Ollama (Llama 3.2)	USE_LOCAL=true
Image generation (N4)	FLUX Kontext Pro	Stable Diffusion 3.5 (local)	USE_LOCAL_IMAGE=true
Audio narration (N6)	Google Cloud TTS	Kokoro TTS (Ollama)	USE_LOCAL_AUDIO=true
FastMCP server	Python process (any env)	Same — no change needed	N/A

7.3 LLM Client Abstraction

```
# config.py
import os
from langchain_anthropic import ChatAnthropic
from langchain_ollama import ChatOllama

def get_llm(capability: str = 'standard'):
    if os.getenv('USE_LOCAL') == 'true':
        model = 'llama3.2' if capability == 'standard' else 'mistral'
        return ChatOllama(model=model, base_url=os.getenv('OLLAMA_URL'))
    if capability == 'reasoning':
        return ChatAnthropic(model='claude-sonnet-4-6')
    return ChatAnthropic(model='claude-haiku-4-5-20261001')
```

7.4 Minimum Hardware for Phase 3

Capability	Minimum spec	Notes
Ollama text LLMs	8 GB RAM, 4-core CPU	Runs Llama 3.2 3B comfortably
Stable Diffusion 3.5	6 GB VRAM GPU	NVIDIA GTX 1660 or equivalent
Kokoro TTS	4 GB RAM	CPU-only, no GPU required
Total for full offline	16 GB RAM, 6 GB VRAM GPU	Mid-range server or workstation

8. Security and Privacy

8.1 API Keys

- All API keys stored as environment variables, never in code or version control
- Anthropic, FLUX (SiliconFlow/Replicate), Google TTS, and Langfuse keys managed via .env
- FastMCP server does not require external API keys — data is local

8.2 Data Handling

- No teacher personal data is stored — inputs (topic, country, grade) are transient
- Generated cartoons and audio are not persisted server-side by default
- Langfuse traces contain prompt inputs — review Langfuse data retention settings for compliance
- Teacher feedback submissions (Phase 2+) should be anonymized before storage

8.3 Content Safety

- LLM-as-judge safety check runs on all panel scripts before image generation
- Image prompts are validated against country avoids list before FLUX API call
- Guardrails AI structural validators run on all structured outputs
- The safety judge prompt is explicitly African-context-aware to prevent false positives on cultural content

9. Deployment

9.1 Development Environment

```
# Clone and install
git clone https://github.com/squad3wakanda/anansi
cd anansi

# Install uv if not already installed
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create virtual environment and install all dependencies
uv sync

# Configure
cp .env.example .env
# Edit .env with your API keys

# Run FastMCP server
uv run python -m anansi.mcp.server

# Run Streamlit UI
uv run streamlit run anansi/ui/app.py
```

9.2 Environment Variables

Variable	Required	Description
ANTHROPIC_API_KEY	Yes (Phase 1)	Claude API key
REPLICATE_API_TOKEN	Yes (Phase 1)	FLUX image generation via Replicate
GOOGLE_APPLICATION_CREDENTIALS	Yes (Phase 1)	Google Cloud TTS service account
LANGFUSE_PUBLIC_KEY	Yes	Langfuse observability public key
LANGFUSE_SECRET_KEY	Yes	Langfuse observability secret key
USE_LOCAL	No (default: false)	Set to 'true' for Ollama text LLMs
OLLAMA_URL	If USE_LOCAL=true	Ollama server URL (default: http://localhost:11434)
MCP_SERVER_URL	No	FastMCP server URL (default: http://localhost:8000)

10. Development Roadmap

10.1 Phase 1 — Cloud MVP

- 5 pilot countries: Kenya, Nigeria, Senegal, Ghana, Cameroon
- All 6 LangGraph nodes implemented and tested
- FastMCP server with static JSON context packs for all 5 countries
- Streamlit UI: input form, progress stream, panel viewer, audio, PDF download
- Langfuse observability fully instrumented from day one
- Pydantic structured outputs at all node boundaries
- LLM-as-judge content safety implemented
- Teacher feedback button (logs to Langfuse, feeds Phase 2 data)

10.2 Phase 2 — Scale

- 15+ countries added based on Phase 1 usage patterns
- Curriculum alignment from aggregated teacher feedback data
- Region-level context packs (sub-national cultural variation)
- FastMCP data layer migrated to SQLite
- Teacher correction submissions via Streamlit UI
- Prompt optimization using Langfuse analytics
- Cost optimization: batch image generation where possible

10.3 Phase 3 — Offline Infrastructure

- Ollama integration: Llama 3.2 and Mistral for all text nodes
- Stable Diffusion 3.5 self-hosted for image generation
- Kokoro TTS for offline audio narration
- Single USE_LOCAL config flag tested in CI from Phase 1
- Deployment guide for school server setup
- Offline-capable FastMCP server (already offline by design)

Anansi — Technical Design Document
SQUAD3 Wakanda · 2026